

Template

```
#include <bits/stdc++.h>
using namespace std;

// ===== Type Aliases =====
using ll = long long;
using pii = pair<int,int>;
using pll = pair<ll,ll>;

// ===== Constants =====
const ll INF = 1e18;
const int MOD = 1e9+7;

// ===== Shortcuts =====
#define all(x) (x).begin(), (x).end()

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n
}
```

Binary Search

```
while (l < r) {
    ll mid = (l + r) / 2;
    if (check(mid))
        r = mid;
    else
        l = mid + 1;
}
```

Prefix Sum Template

```
vector<ll> pref(n+1, 0);
for (int i = 1; i <= n; i++) {
    pref[i] = pref[i-1] + a[i];
}
// sum from l to r (1-indexed)
ll sum = pref[r] - pref[l-1];
```

Sliding Window Skeleton

```
int l = 0;
for (int r = 0; r < n; r++) {
    // add a[r]
```

```

    while (/* condition invalid */) {
        // remove a[l]
        l++;
    }
    // update answer
}

```

Sorting

```

vector<int> a = {5,2,9,1,5,6};
// Sort ascending
sort(a.begin(), a.end());
// Sort descending
sort(a.begin(), a.end(), greater<int>());
// Sort pairs by first, then second
vector<pair<int,int>> p = {{3,4},{2,5},{3,1}};
sort(p.begin(), p.end()); // default: first ascending, tie → second ascending
// Custom comparator
sort(p.begin(), p.end(), [](pair<int,int> x, pair<int,int> y){
    return x.second < y.second; // sort by second
});

```

Greedy

```

sort(a.begin(), a.end());

for(auto x : a){
    if(valid(x))
        take(x);
}

```

1 Vectors / Arrays

```
vector<int> v;           // declare vector
v.push_back(x);         // add element at end
v.pop_back();           // remove last element
v.size();               // number of elements
v.empty();              // check if empty
sort(all(v));           // sort ascending
sort(all(v), greater<int>()); // sort descending
reverse(all(v));        // reverse vector
*max_element(all(v))    // maximum element
*min_element(all(v))    // minimum element
accumulate(all(v), 0LL) // sum of elements, use 0LL for long long
fill(v.begin(), v.end(), 0) // fill vector with 0
iota(v.begin(), v.end(), 1) // fill vector with 1,2,3...

v.erase(v.begin() + i); // remove element by index
v.insert(v.begin() + i, x); // insert element

int idx = lower_bound(X, X+k, A[i]) - X; // Subtract X to get index (arrays
decay to pointers)
```

2 Pairs

```
pair<int,int> p = {a,b}; // declare pair
p.first, p.second        // access elements
swap(p.first, p.second)  // swap elements
tie(a,b) = p             // unpack pair
make_pair(x,y)           // create pair
vector<pair<int,int>> vp;
sort(vp.begin(), vp.end()); // sort by first, tie → second
sort(vp.begin(), vp.end(), [](auto &a, auto &b){ return a.second < b.second; });
// sort by second
```

3 Sets / Multisets

```
set<int> s; // unique elements, sorted
s.insert(x); // add element
s.erase(x); // remove element
s.count(x); // check if element exists (0 or 1)
auto it = s.lower_bound(x); // first element >= x
auto it = s.upper_bound(x); // first element > x

multiset<int> ms; // allows duplicates
ms.insert(x);
ms.erase(ms.find(x)); // erase single occurrence
```

4 Maps / Unordered Maps

```
map<int,int> m; // key → value, ordered
unordered_map<int,int> um; // key → value, hash table (faster)
m[key] = value;
m.count(key); // check if key exists
m.erase(key);
auto it = m.find(key); // iterator to key or end
```

5 Priority Queue (Heap)

```
priority_queue<int> pq; // max-heap
pq.push(x);
pq.top(); // max element
pq.pop(); // remove max

priority_queue<int, vector<int>, greater<int>> pq_min; // min-heap

priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>> pqp;
// min-heap pair
```

6 Common Functions / Helpers

```
all(v) // shortcut: v.begin(), v.end()
lower_bound(all(v), x) // first element >= x (sorted vector)
upper_bound(all(v), x) // first element > x (sorted vector)
int idx = max_element(all(v)) - v.begin(); // return index
int idx = min_element(all(v)) - v.begin(); // return index
binary_search(all(v), x) // true if x exists in sorted vector
count(all(v), x) // count occurrences
clamp(x, lo, hi) // clamp x between lo and hi (C++17)
gcd(a,b) // greatest common divisor
lcm(a,b) // least common multiple (C++17)
abs(x) // absolute value
max(a,b), min(a,b) // max/min of two numbers
swap(a,b) // swap values
reverse(all(v)) // reverse container
sort(all(v)); //remove duplicates
v.erase(unique(all(v)), v.end()); //remove duplicates
auto it = find(all(v), x);
```

7 Iteration Tricks

```
for(auto x : v) cout << x << " "; // range-based for
for(int i=0; i<n; i++) { ... } // classic for loop
for(int i=n-1; i>=0; i--) { ... } // reverse loop
for(auto &[a,b] : vp) { ... } // loop through vector of pairs

cerr << x << "\n"; //use to debuged
```